obra/superpowers/**SKILL.md**

---

name:

SUPER—POWERS

description: Skills para trabalhar com agentes de IA de forma verificável

author: **Regis Araujo Melo**

Agentes & Skills para desenvolvimento · skills.sh/obra



QUEM APRESENTA

REGIS ARAUJO MELO

Engenheiro iOS Sênior

Desenvolvedor iOS sênior com mais de 20 anos em tecnologia, com foco em apps nativos para os setores financeiro, varejo e mobilidade. Arquitetura modular, segurança, acessibilidade e performance, evolução de código legado e entrega contínua.

LinkedIn
linkedin.com/in/regisaraujomelo

ROTEIRO

60 MINUTOS, 25 DE PRÁTICA

0:00–0:15

Conceitos

problema, agente, skill e o fluxo mínimo

0:15–0:40

Laboratório

esclarecer, planejar e testar primeiro, no seu repositório

0:40–1:00

Escala, limites e perguntas

o que vem depois do primeiro teste

O PROBLEMA

RÁPIDO NÃO É VERIFICÁVEL

Sem processo, um agente pode:

- Implementar antes de esclarecer o requisito
- Escrever o teste depois, só para passar
- Dizer “pronto” sem evidência

E se o processo viesse junto com o agente?

CONCEITO

AGENTE = MODELO + HARNESS



Modelo (LLM)
Gera texto e decide o próximo passo. Sozinho, não age.

Contexto
O que o modelo vê agora. Tem limite.

Harness
Roda o loop, pede a sua permissão e carrega as skills.

Ferramentas · MCP
As mãos do agente; MCP é o padrão para conectar serviços externos.

No chat, **o harness é você**: você copia, cola e executa.

Mais capacidade pede mais processo e controle.

CONCEITO

SKILL É UM PROCESSO EM ARQUIVO

Uma pasta com um `SKILL.md`: nome, descrição e instruções. O harness põe **só a descrição** no contexto; as instruções entram **quando a skill é usada**, pela descrição ou quando você a cita (`/nome`).

```
---
name: brainstorming
description: Use antes de qualquer
  trabalho criativo…
---
```

Isto fica sempre no contexto. O resto do arquivo, só quando precisa.

Prompt
Você repete a cada conversa.

Ferramenta · MCP
Dá ao agente uma capacidade: ler arquivo, rodar comando, acessar um serviço.

Skill
Dá ao agente um método: versionado e revisável como código.

O FLUXO MÍNIMO

QUATRO SKILLS PARA O FLUXO MÍNIMO

01

brainstorming

requisito e design aprovados

→

02

writing-plans

plano em passos pequenos

→

03

test-driven-development

teste vermelho, depois verde

→

04

verification-before-completion

“pronto” só com evidência

O pacote tem 14 skills. Hoje usamos estas quatro; o catálogo completo está no apêndice.

DEFINIÇÃO

CONFIÁVEL = EVIDÊNCIA OBSERVÁVEL

ANTES

- Requisito esclarecido
- Plano revisável

DURANTE

- Teste falhando antes do código
- Teste passando depois

DEPOIS

- Diff revisado por você
- Resultado verificado por você

Sem essas evidências, é só código que parece certo.

BRAINSTORMING NA PRÁTICA

O AGENTE PERGUNTA ANTES DE CODAR

No caso da lista de bancos, cada resposta sua vira requisito escrito. A saída é um design curto, **aprovado por você**.

Exemplo ilustrativo, não é saída real do agente.

```
agente › Como a lista deve ser ordenada?
você   › Por nome, sem diferenciar maiúsculas.
agente › A busca vale para nome, código ou os
dois?
você   › Os dois. Código só por prefixo.
agente › “credito” deve encontrar “Crédito”?
```

TAREFA PRÉVIA

PRONTO PARA COMEÇAR?

Login — Abra o `copilot` e confira que está autenticado (`/login`).

Instalação

```
npx skills add obra/superpowers -g -a github-copilot
```

Ou como plugin do Copilot CLI: comandos nos pré-requisitos.

Conferir — `/skills list` mostra `brainstorming`. Escreva **pronto** no chat.

Sem ambiente? Forme dupla ou acompanhe a tela do apresentador.

ANTES DE COMEÇAR

REGRAS DO LABORATÓRIO

Branch local, sem push

`workshop/seu-nome`, com o push bloqueado. Os comandos estão no guia.

Aprove comando a comando

Leia cada comando antes de permitir. Nunca “permitir tudo”.

Nada sensível

Não cole segredos, tokens nem dados de clientes. Não peça para ler arquivos de credenciais.

Os pedidos estão no guia do participante, no chat. Copie de lá, não da tela.

LABORATÓRIO · ETAPA 1 DE 3

8 min

ESCLARECER

1. Crie a branch e abra o `copilot` na raiz do repositório.
2. Cole o pedido da etapa 1 do guia (`/brainstorming`, no máximo 3 perguntas).
3. **Responda e aprove** o design.

A HISTÓRIA E O RECORTE

Como cliente que quer trazer investimentos de outro banco, **quero** escolher o banco de origem numa lista, **para** iniciar a portabilidade sem digitar dados do banco. **Recorte:** uma função que ordena por nome e filtra por nome ou código.

{"code": "001", "institutionType": "1", "name": "Banco do Brasil", "cnpj": "00000000000191"}

Checkpoint no chat: a pergunta mais útil que o agente fez. Passou de 8 min? Use o design pronto do guia.

LABORATÓRIO · ETAPA 2 DE 3

4 min

PLANEJAR

1. Peça o plano com `/writing-plans` (pedido da etapa 2).
2. No máximo **3 tarefas**.
3. Confira que a Tarefa 1 começa por um teste.

Checkpoint no chat: quantas tarefas o seu plano tem. Travou? Use o plano pronto do guia.

LABORATÓRIO · ETAPA 3 DE 3

11 min

VERMELHO, DEPOIS VERDE

1. Execute só a Tarefa 1 com `/test-driven-development`.
2. Rode o teste **você mesmo**: falha, depois passa.
3. Revise o diff e faça commit só local.

Checkpoint no chat: a saída do teste verde. Não terminou? O guia mostra como continuar.

RETROSPECTIVA

O QUE AS EVIDÊNCIAS MOSTRARAM

Quantos chegaram ao verde?

Contagem pelos checkpoints do chat.

A pergunta mais útil

Que requisito ela esclareceu antes do código?

Onde travou

Travar também é dado: mostra onde o processo protege.

ESCALA, REVISÃO E ENTREGA

DEPOIS DO PRIMEIRO TESTE

01

`executing-plans`

seguir o plano tarefa a tarefa; com `subagent-driven-development`, onde houver suporte a subagentes

02

`requesting-code-review`

revisão independente antes de integrar

03

`finishing-a-development-branch`

merge, PR ou descarte, com os testes verdes

COM OS PÉS NO CHÃO

LIMITES E CONTROLES

Custa tempo e tokens

Use o fluxo completo onde há dúvida ou risco. Correção de uma linha não precisa.

Skill de terceiros

É instrução para o seu agente. Revise o `SKILL.md` e os scripts antes de instalar.

Você continua responsável

O agente não substitui revisão, testes nem a sua assinatura no PR.

DEPOIS DO WORKSHOP

PRÓXIMO PASSO: UMA HISTÓRIA REAL

1. **Termine o laboratório** com a seção “Como continuar” do guia.
2. Use o fluxo numa **história pequena** desta sprint.
3. Traga as **seis evidências** na próxima reunião.

Perguntas?

APÊNDICE

MATERIAL DE APOIO

Para usar se sobrar tempo ou surgir uma pergunta.

1. **O mapa das 14 skills** e os quatro grupos
2. **brainstorming**, **writing-plans** e **executing-plans** em detalhe
3. **O fluxo completo** em sete etapas
4. **SDD**: Spec-Driven Development e o Spec Kit
5. **obra, skills.sh** e skills por plataforma

SPEC-DRIVEN DEVELOPMENT

SDD: A SPEC VEM PRIMEIRO

"Definir **o quê e o porquê** antes de decidir **o como**."

01
spec
o quê e o porquê: requisitos e critérios de aceite

→

02
plano
o como: decisões técnicas a partir da spec

→

03
tarefas
passos pequenos e verificáveis

→

04
código
implementado e verificado contra a spec

Definição: README do **github/spec-kit**, consultado em 19/09/2026.

SPEC-DRIVEN DEVELOPMENT

O FLUXO QUE PRATICAMOS JÁ É SDD

ETAPA DO SDD	SKILL DO SUPERPOWERS	EVIDÊNCIA
Especificar	`brainstorming`	design aprovado em `docs/superpowers/specs/`
Planejar e quebrar em tarefas	`writing-plans`	plano com tarefas pequenas
Implementar	`test-driven-development`, `executing-plans`	teste vermelho, depois verde
Verificar	`verification-before-completion`	resultado verificado contra a spec

SPEC-DRIVEN DEVELOPMENT

SPEC KIT: SDD COMO FERRAMENTA

`/speckit-constitution`	princípios do projeto que toda spec deve respeitar
`/speckit-specify`	a spec: o quê e o porquê
`/speckit-plan`	o plano técnico: o como
`/speckit-tasks`	tarefas a partir do plano
`/speckit-implement`	implementa as tarefas; repete com **`/speckit-converge`** até convergir

GitHub Spec Kit (**`github/spec-kit`**), com integração padrão para o GitHub Copilot. Comandos conforme o README em 19/09/2026: confira a versão antes de usar.

O MAPA DO SUPERPOWERS

14 SKILLS, 4 MOMENTOS DO TRABALHO

01 Pensar antes de codar

- brainstorming
- writing-plans
- executing-plans

02 Qualidade e disciplina

- test-driven-development
- systematic-debugging
- verification-before-completion

03 Escala com agentes

- subagent-driven-development
- dispatching-parallel-agents
- using-git-worktrees

04 Revisão e entrega

- requesting-code-review
- receiving-code-review
- finishing-a-development-branch
- writing-skills
- using-superpowers

uso no dia a dia | sugestão

01

PENSAR ANTES DE CODAR

Objetivo: entender e planejar antes de codar.

SEM A SKILL	COM A SKILL, O AGENTE…
`brainstorming` Pedido vago vira código errado.	faz perguntas uma a uma, propõe 2 ou 3 abordagens e só avança com um design aprovado.
`writing-plans` Tarefa grande vira improviso.	quebra o trabalho em passos pequenos, com arquivos, testes e critério de pronto.
`executing-plans` O plano é esquecido no meio do caminho.	executa o plano em lotes, com pontos de revisão.

02

QUALIDADE E DISCIPLINA

SEM A SKILL	COM A SKILL, O AGENTE…
`test-driven-development`	
Teste escrito depois só confirma o código.	segue vermelho, verde, refatorar: o teste falha primeiro.
`systematic-debugging`	
O bug é “consertado” no chute.	reproduz, isola e encontra a causa raiz antes de corrigir.
`verification-before-completion`	
“Pronto!” sem prova.	só declara pronto com evidência: teste rodado, saída conferida.

Objetivo: cada afirmação acompanhada de evidência.

03

ESCALA COM AGENTES

Objetivo: paralelizar com revisão entre tarefas.

SEM A SKILL	COM A SKILL, O AGENTE…
`subagent-driven-development`	
Um agente só se perde num contexto longo.	entrega cada tarefa do plano a um subagente novo, com revisão entre tarefas, onde houver suporte.
`dispatching-parallel-agents`	
Problemas independentes resolvidos em fila.	dispara vários agentes em paralelo, um por problema.
`using-git-worktrees`	
Tarefas paralelas se atropelam no mesmo diretório.	isola cada frente de trabalho na sua própria worktree.

04

REVISÃO E ENTREGA

Objetivo: revisar, integrar e registrar o que se aprendeu.

SEM A SKILL	COM A SKILL, O AGENTE…
`requesting-code-review`	
O agente revisa o próprio trabalho, com o próprio viés.	pede uma revisão independente antes de integrar.
`receiving-code-review`	
Aceita toda sugestão sem pensar.	avalia cada sugestão com rigor técnico e contesta quando necessário.
`finishing-a-development-branch`	
Branch pronta e esquecida.	apresenta opções claras: merge, PR ou limpeza.
`writing-skills`	
O time aprende algo e o conhecimento se perde.	transforma o aprendizado numa nova skill, testada.
`using-superpowers`	
O agente não sabe que as skills existem.	procura e usa a skill certa antes de agir. É a porta de entrada.

PENSAR ANTES DE CODAR · EM DETALHE

brainstorming

QUANDO USAR

Antes de qualquer trabalho criativo: uma feature nova, um componente, uma mudança de comportamento.

REGRA DE OURO

> **Nenhum código antes de um design aprovado. Vale até para o que "parece simples".**

O QUE GERA

`docs/superpowers/specs/AAAA-MM-DD-tema-design.md`

Na prática: na história da lista de bancos, o agente pergunta como ordenar, se a busca é por nome ou código e o que mostrar com a lista vazia, antes de escrever código.

O PROCESSO

1. **Explora o contexto:** arquivos, docs e commits recentes.
2. **Pergunta uma coisa por vez,** de preferência com múltipla escolha.
3. **Propõe 2 ou 3 abordagens** com prós, contras e uma recomendação.
4. **Apresenta o design em partes** e pede aprovação de cada uma.
5. **Escreve o spec e faz commit,** depois revisa o texto e pede a revisão de quem pediu.
6. **Passa a vez para** `writing-plans`.

PENSAR ANTES DE CODAR · EM DETALHE

writing-plans

QUANDO USAR

Com um spec aprovado em mãos, antes de tocar no código.

REGRA DE OURO

> **Escrever para um dev que não conhece o projeto: nada de “TODO” ou “implementar depois”.**

O QUE GERA

`docs/superpowers/plans/AAAA-MM-DD-feature.md`

Na prática: a lista de bancos vira tarefas como “ordenar por nome + teste”, “filtrar por nome + teste” e “filtrar por código + teste”.

O PROCESSO

1. **Mapeia os arquivos** e a responsabilidade de cada um.
2. **Quebra em tarefas pequenas:** cada passo leva de 2 a 5 minutos.
3. **Em cada tarefa:** teste que falha, implementação, teste passando, commit.
4. **Escreve tudo por extenso:** caminhos exatos, código completo, comando e saída esperada.
5. **Revisa o plano contra o spec:** cobertura, pendências e nomes consistentes.
6. **Oferece a execução:** com subagentes ou na própria sessão.

PENSAR ANTES DE CODAR · EM DETALHE

executing-plans

QUANDO USAR

Quando há um plano escrito para executar com pontos de revisão. Com subagentes disponíveis, prefira `subagent-driven-development`.

REGRA DE OURO

> **Travou? Para e pergunta. Não adivinha e não força o caminho.**

O QUE GERA

Código commitado num branch, pronto para integrar.

Na prática: cada tarefa da lista de bancos roda na branch `workshop/seu-nome`; se o teste da ordenação falhar sem motivo claro, o agente para e pergunta.

O PROCESSO

1. **Prepara um espaço isolado** com `using-git-worktrees`.
2. **Lê o plano com olhar crítico** e levanta dúvidas antes de começar.
3. **Executa tarefa por tarefa:** segue os passos à risca e roda as verificações.
4. **Para no primeiro bloqueio:** dependência faltando, teste falhando, instrução confusa.
5. **Fecha com `finishing-a-development-branch`**: merge, PR ou limpeza.

JUNTANDO TUDO

O FLUXO COMPLETO

01 brainstorming › 02 writing-plans › 03 using-git-worktrees › 04 subagent-driven-development + test-driven-development › 05 requesting-code-review › 06 verification-before-completion › 07 finishing-a-development-branch

COMO COMEÇAR

```
# padrão do skills.sh
$ npx skills add obra/superpowers -g -a github-copilot
# ou como plugin do Copilot CLI
$ copilot plugin marketplace add obra/superpowers-marketplace
$ copilot plugin install superpowers@superpowers-marketplace
```

Depois, dentro do **copilot**: **/skills list**

skills.sh/obra · github.com/obra/superpowers

Confiável = evidência observável.

AUTOR E CATÁLOGO

UM CATÁLOGO DE MÉTODOS

obra

Jesse Vincent publica skills que codificam práticas de engenharia sênior para agentes.

skills.sh

Diretório aberto de skills: você encontra, instala e usa no seu agente.

OUTROS PACOTES DO AUTOR

episodic-memory	o agente lembra de conversas anteriores
the-elements-of-style	escrita clara e concisa
superpowers-lab	experimentos: tmux, funções duplicadas…
superpowers-chrome	navegar na web pelo Chrome

O principal é o **superpowers** →

SKILLS POR PLATAFORMA · 1 DE 3

iOS · SWIFT

`swiftui-pro`
`twostraws/swiftui-agent-skill`

Revisa código SwiftUI: APIs modernas, manutenção e performance.

`swiftui-expert-skill`
`avdlee/swiftui-agent-skill`

Escreve e refatora SwiftUI: estado com `@Observable`, composição de views e listas.

`swift-concurrency`
`avdlee/swift-concurrency-agent-skill`

Diagnostica concorrência, troca callbacks por async/await e guia a migração para o Swift 6.

`swift-testing-pro`
`twostraws/swift-testing-agent-skill`

Escreve e revisa testes com o framework Swift Testing.

`ios-accessibility`
`dpearson2699/swift-ios-skills`

Constrói e audita acessibilidade: VoiceOver, Voice Control e Dynamic Type.

COMBINA COM O SUPERPOWERS

`swift-testing-pro` + `test-driven-development`: o teste vem primeiro, já com as APIs do Swift Testing.

No Copilot CLI: copie a pasta da skill para **`.github/skills/`** ou **`~/.copilot/skills/`**

Revise o **`SKILL.md`** antes de instalar.

SKILLS POR PLATAFORMA · 2 DE 3

ANDROID · KOTLIN

migrate-xml-views-to-jetpack-compose

android/skills oficial

Roteiro guiado para migrar telas em XML para Jetpack Compose.

navigation-3

android/skills oficial

Instala e migra para o Navigation 3: deep links, múltiplas pilhas e diálogos.

edge-to-edge

android/skills oficial

Adapta apps Compose para a tela de borda a borda e corrige os problemas comuns.

kotlin-tooling-java-to-kotlin

kotlin/kotlin-agent-skills JetBrains

Converte código Java em Kotlin idiomático.

kotlin-coroutines-flows

affaan-m/ecc

Coroutines e Flow: concorrência estruturada, StateFlow, erros e testes.

COMBINA COM O SUPERPOWERS

`migrate-xml-views-to-jetpack-compose` + `writing-plans`: a migração vira um plano tela a tela.

No Copilot CLI: copie a pasta da skill para **`.github/skills/`** ou **`~/.copilot/skills/`**

Revise o **`SKILL.md`** antes de instalar.

SKILLS POR PLATAFORMA · 3 DE 3

BACKEND · JAVA

java-springboot

github/awesome-copilot GitHub

Boas práticas para aplicações Spring Boot.

create-spring-boot-java-project

github/awesome-copilot GitHub

Gera o esqueleto de um projeto Spring Boot em Java.

java-junit

github/awesome-copilot GitHub

Testes com JUnit 5, incluindo testes orientados a dados.

springboot-security

affaan-m/ecc

Spring Security: autenticação, autorização, validação, CSRF, segredos e rate limiting.

springboot-tdd

affaan-m/ecc

TDD com JUnit 5, Mockito, MockMvc, Testcontainers e JaCoCo.

COMBINA COM O SUPERPOWERS

`springboot-security` + `requesting-code-review`: segurança revisada por um olhar independente antes do merge.

No Copilot CLI: copie a pasta da skill para **`.github/skills/`** ou **`~/.copilot/skills/`**

Revise o **`SKILL.md`** antes de instalar.